

Product Requirements Documents

AI Engineer Portfolio — Projects 1–10 (Agentic Workflows + RAG)

Scoped to be buildable solo, each forcing real design decisions (not framework defaults) so you can defend them in an interview. Each follows the same structure for consistency.

PRD 1 — Support-Ticket Triage Agent

Overview

An agent that classifies incoming support tickets, retrieves related past tickets/docs, drafts a response, and escalates ambiguous cases to a human.

Problem Statement

Manual ticket triage is slow and inconsistent. An agent that reliably handles the easy 70–80% and cleanly escalates the rest proves you can own a real automatable business process end-to-end, not just a chatbot demo.

Goals

Achieve a measured task-success rate with a defined escalation path for low-confidence cases.

Produce a full, auditable decision trace per ticket.

Handle tool failures (e.g., a lookup API timing out) without silently dropping the ticket.

Non-Goals

Actually sending customer-facing replies unsupervised (always human-approved for v1).

Multi-language support (English-only for v1, documented as a limitation).

Users & Personas

Persona	Needs
Support agent	Reviews agent-drafted responses, approves/edits/escalates
Team lead	Views triage accuracy and escalation-rate dashboards

Tech Stack

Layer	Choice	Notes
Language/runtime	Python 3.11+	Matches your existing LangGraph/RAG repos
Orchestration	LangGraph	Planner/executor graph with an explicit escalation-gate node
Backend API	FastAPI	Consistent with <code>production-grade-rag-engine</code>
LLM provider	OpenAI or Anthropic API, behind a thin provider adapter	Swappable without touching agent logic
Retrieval / vector DB	Qdrant	Reuses your existing RAG-engine ingestion pipeline for ticket-search
Data store	PostgreSQL	Tickets, traces, escalation state
Async/background jobs	Inngest (or Celery + Redis)	Tool-call retries, background classification
Observability & eval	OpenTelemetry traces + a custom eval harness (task-success rate, escalation accuracy)	Purpose-built, not RAGAS (this is agent-behavior eval, not retrieval eval)
CI/CD	GitHub Actions	Runs the labeled-ticket eval set on every PR
Testing	Pytest + a labeled 50-100 ticket eval fixture	Eval set is checked into the repo, not external
Deployment	Docker + a lightweight review UI (Streamlit or a small Next.js app)	Human-approval queue for drafted responses

Functional Requirements

- FR1: Ingest a ticket (public dataset or your own inbox/GitHub Issues) and classify category + urgency.
- FR2: Retrieve related past tickets/docs via RAG before drafting a response.
- FR3: Draft a response with citations to retrieved sources.
- FR4: Route to human review when confidence is below a tunable threshold.
- FR5: Persist a full step-by-step trace (classification → retrieval → draft → routing decision) per ticket.
- FR6: Detect and break out of agent loops (e.g., repeated failed tool calls) rather than looping indefinitely.

Non-Functional Requirements

Category	Requirement
Reliability	Tool-call failures trigger retry with backoff, then graceful escalation
Auditability	Every decision traceable end-to-end with timestamps
Latency	p95 triage time under a defined budget (e.g., 30s)
Cost	Per-ticket cost tracked and reported

Architecture Notes

Planner/executor split (or single ReAct loop for v1) using LangGraph; tools: ticket-search (your RAG index), classification call, draft-generation call; state persisted in Postgres; escalation gate as an explicit graph node, not a prompt instruction alone.

API Surface (representative)


POST /tickets/:id/triage
GET /tickets/:id/trace
GET /dashboard/triage-metrics

Milestones

- Single-step classifier only, no retrieval.
- Add RAG retrieval + draft generation.
- Add confidence-based escalation + human review UI.
- Add loop detection, failure handling, and full tracing.
- Load test + report task-success rate on a labeled test set.

Success Metrics

- Task success rate (%) on a held-out labeled set of tickets.
- Escalation rate and false-escalation rate (easy tickets wrongly escalated).
- Mean/median steps to completion.

Risks

- Labeled test-set construction is manual work — budget real time for it; a small (50–100 ticket) hand-labeled set is enough to be credible.

PRD 2 — Code-Review Agent

Overview

An agent that performs automated first-pass review on pull requests — style, obvious bugs, missing tests, security smells — with comments posted directly to the PR.

Problem Statement

Human reviewers waste time on issues a tool could catch first. This agent proves you can build reliable dev-tools AI, one of the hottest 2026 YC verticals, integrated with a real external system (GitHub) rather than a mocked tool.

Goals

- Reduce reviewer time-to-first-comment on obvious issues.
- Keep false-positive comment rate low enough to avoid reviewer fatigue.
- Integrate with real GitHub webhooks, not a simulated PR feed.

Non-Goals

- Full semantic understanding of complex architectural changes (flag those for human review instead).
- Auto-merging or auto-approving PRs.

Users & Personas

Persona	Needs
PR author	Gets fast, useful automated feedback before human review
Reviewer	Sees agent comments already resolved/triaged, focuses on substance

Tech Stack

Layer	Choice	Notes
Language/runtime	Python 3.11+	Worker/agent logic
Orchestration	A lean tool-calling loop (LangGraph optional; may be overkill for a 2-tool agent)	Keep this one simple — the value is the GitHub integration, not agent complexity
Backend API	FastAPI	Webhook receiver
LLM provider	OpenAI or Anthropic API	Semantic review pass
Static-analysis tools	ESLint/Semgrep (or language-appropriate equivalent) invoked as agent tools	Not LLM-based — deterministic first pass

Layer	Choice	Notes
GitHub integration	A GitHub App using Octokit (Node) or PyGithub, wrapped in an adapter with timeout/retry	Per external-integration adapter pattern
Data store	PostgreSQL	Review records, human feedback ("not useful" flags)
Async/background jobs	Redis + BullMQ (or Celery)	One job per PR event
Observability & eval	OpenTelemetry + a false-positive-rate dashboard	Tracks reviewer dismissal rate directly
CI/CD	GitHub Actions	Also dogfoods the agent on its own PRs if desired
Testing	Pytest + integration tests against a forked test repo	Never test-comment on the live upstream repo
Deployment	Docker, webhook endpoint on a managed host	Needs a public HTTPS endpoint for GitHub webhooks

Functional Requirements

- FR1: Receive a GitHub PR webhook and pull the diff.
- FR2: Run static-analysis tool calls (lint, basic security scan) as agent tools.
- FR3: Run an LLM-based semantic review pass for logic/style issues static tools miss.
- FR4: Post comments to the PR with a confidence score per comment.
- FR5: Allow a human to mark a comment as "not useful" — log this feedback for later analysis.
- FR6: Skip or flag (not block) PRs above a diff-size threshold where review quality degrades.

Non-Functional Requirements

Category	Requirement
Reliability	Webhook processing is idempotent under GitHub's at-least-once delivery
Latency	Comments posted within a defined SLA (e.g., 3 min) of PR open/update
Cost	Per-PR review cost tracked
Precision	False-positive rate on flagged issues measured against human dismissal rate

Architecture Notes

NestJS/FastAPI webhook receiver → job queue → worker running static-analysis tool calls + LLM semantic pass → GitHub API comment-posting adapter with retry/timeout handling.

API Surface (representative)


POST /webhooks/github/pr
GET /reviews/:pr_id
GET /dashboard/review-metrics

Milestones

Webhook receiver + diff fetch, no analysis yet.
Static-analysis tool integration + comment posting.
LLM semantic review pass added.
Confidence scoring + human feedback capture.
Metrics dashboard: false-positive rate, time-to-comment.

Success Metrics

False-positive rate (% of comments marked "not useful").
Mean time-to-first-comment vs. human baseline.
% of PRs where agent caught an issue a human confirmed was real.

Risks

Needs a real (or realistic fork of a) public repo with genuine PR activity to generate credible metrics — pick an active open-source project you can safely comment-test against (a fork, not the live upstream).

PRD 3 — Compliance Document Reviewer

Overview

An agent that checks contracts/policy documents against a defined rule set (e.g., GDPR clauses, internal policy) and flags violations with citations, routing high-risk flags to mandatory human review.

Problem Statement

Compliance review is slow, manual, and inconsistent across reviewers. This project proves you can build guardrail-aware, citation-grounded AI for a genuinely high-stakes domain — a strong signal for EU employers given the EU AI Act.

Goals

Every flag must cite the exact clause/passage that triggered it — no unexplained flags.
High-severity flags always route to mandatory human review (never auto-resolved).
Measured false-positive rate reported explicitly, not hidden.

Non-Goals

Providing actual legal advice or final compliance sign-off (always human-owned).
Covering more than one rule set / jurisdiction for v1.

Users & Personas

Persona	Needs
Compliance reviewer	Reviews flagged clauses with full citation context
Document owner	Uploads documents, sees review status

Tech Stack

Layer	Choice	Notes
Language/runtime	Python 3.11+	
Orchestration	Custom rule-engine + LLM hybrid pipeline (plain function pipeline, not full agent framework)	Severity routing is rule-based on purpose — not an LLM decision
Backend API	FastAPI	
LLM provider	OpenAI or Anthropic API	Used only for the semantic-check layer, not the routing gate
Rule store	Structured, versioned rules in PostgreSQL	Not hardcoded in prompts — rules are data
Retrieval / vector DB	Qdrant or pgvector	Clause-level embeddings for citation grounding
Data store	PostgreSQL	Documents, flags, immutable audit log
Async/background jobs	Celery + Redis	Document ingestion and chunk-level checks
Observability & eval	Precision/recall dashboard + full audit trail via structured logs	Explicitly reports false-positive rate, per NFR

Layer	Choice	Notes
CI/CD	GitHub Actions	Runs the labeled-document eval set on every PR
Testing	Pytest + a labeled test-document set (built from one public regulation)	
Deployment	Docker + a Streamlit admin console for reviewers	

Functional Requirements

- FR1: Upload a document; parse into clause/section-level chunks.
- FR2: Check each chunk against a defined rule set (rules stored as structured, versioned data).
- FR3: Flag violations with a citation to the exact source passage.
- FR4: Route flags above a severity threshold to mandatory human review before any downstream action.
- FR5: Maintain a full audit log of every review decision (agent flag + human resolution).

Non-Functional Requirements

Category	Requirement
Explainability	100% of flags carry a traceable citation
Safety	High-severity flags cannot be auto-resolved by the agent
Precision	False-positive rate measured against a labeled test document set
Auditability	Full, immutable audit trail per document

Architecture Notes

Document parsing → clause-level chunking → rule-engine + LLM hybrid check per chunk → citation-grounded flag generation → severity-based routing (rule-based gate, not LLM-decided) → audit log persisted in Postgres.

API Surface (representative)


POST /documents
GET /documents/:id/flags
POST /flags/:id/resolve

Milestones

Document ingestion + clause chunking.
Rule engine + LLM hybrid check, single rule set.
Citation-grounded flag generation.
Severity-based human-review routing.
Audit log + false-positive rate evaluation on a labeled document set.

Success Metrics

Precision/recall on a labeled test-document set (using a public policy/regulation text as the rule source).
% of flags with a valid, verifiable citation (target: 100%).
False-positive rate, reported honestly.

Risks

Rule-set design is the hardest part — start narrow (e.g., 5–10 concrete rules from one public regulation) rather than attempting broad coverage.

PRD 4 — Multi-Agent Research Assistant

Overview

An agent system that decomposes a research question into sub-questions, researches each in parallel via specialized sub-agents, and synthesizes a cited report.

Problem Statement

Most portfolio "agent" projects are a single loop calling 2–3 tools sequentially. This project proves you can design genuine multi-agent coordination — parallel delegation, source deduplication, and synthesis — which is a meaningfully harder and more valuable skill.

Goals

Demonstrate real parallel sub-agent execution, not sequential calls dressed up as "multi-agent."
Produce a synthesized report with every claim traceable to a source.
Deduplicate overlapping findings across sub-agents before synthesis.

Non-Goals

Real-time web research at unlimited scope (bound to a fixed, reasonable source set or search budget per query).
Long-running (multi-hour) research sessions for v1.

Users & Personas

Persona	Needs
Requester Submits a research question, receives a structured, cited report	

Tech Stack

Layer	Choice	Notes
Language/runtime	Python 3.11+	
Orchestration	LangGraph supervisor/worker pattern with fan-out and join nodes	The genuine parallelism is the point of this project — verify it's real, not simulated sequential calls
Backend API	FastAPI	
LLM provider	OpenAI or Anthropic API	
Search tools	Tavily or SerpAPI (web search) + Qdrant (internal doc search)	Bounded per-query search budget to control cost
Data store	PostgreSQL	Research traces, synthesized reports
Async/concurrency	Native async fan-out within LangGraph; Redis only if sessions need to persist across long-running queries	Keep this lightweight — avoid a queue just to look "production"
Observability & eval	Per-sub-agent cost/latency tracking, citation-integrity checker	Wall-clock parallel-vs-sequential benchmark is a first-class metric here
CI/CD	GitHub Actions	
Testing	Pytest + a benchmark harness comparing parallel vs. sequential execution	
Deployment	Docker + a simple report-viewer UI (Streamlit or Next.js)	

Functional Requirements

- FR1: Supervisor agent decomposes a question into 3–5 sub-questions.
- FR2: Sub-agents research each sub-question in parallel (web search and/or document search tools).
- FR3: Source deduplication step before synthesis (same source cited once, not per sub-agent).
- FR4: Synthesis agent produces a structured report with inline citations.
- FR5: Full trace of decomposition → sub-agent results → synthesis is persisted and viewable.

Non-Functional Requirements

Category	Requirement
Parallelism	Sub-agents measurably execute concurrently (wall-clock time < sum of sequential times)
Cost	Per-query cost tracked and capped with a hard budget
Citation integrity	Every synthesized claim traceable to at least one retrieved source

Architecture Notes

LangGraph supervisor/worker pattern with fan-out to parallel sub-agent nodes, a join node for deduplication, and a final synthesis node; source-dedup via URL/content-hash matching before synthesis.

API Surface (representative)

POST /research GET /research/:id/report GET /research/:id/trace
--

Milestones

- Single-agent sequential research baseline (for comparison).
- Supervisor decomposition + parallel sub-agent execution.
- Source deduplication step.
- Synthesis with inline citations.
- Benchmark: parallel vs. sequential wall-clock time and cost comparison.

Success Metrics

- Wall-clock speedup of parallel vs. sequential execution.
- % of synthesized claims with a valid citation.
- Cost per query, with budget-cap enforcement demonstrated.

Risks

- Search-tool rate limits can throttle "true" parallelism — document this constraint honestly rather than overclaiming parallel speedup.

PRD 5 — Autonomous Data-Pipeline Debugger

Overview

An agent that diagnoses a broken ETL/data pipeline by inspecting logs, checking recent schema changes, testing hypotheses, and proposing a fix — an AI-for-ops/SRE project.

Problem Statement

Most agent portfolios target customer-facing workflows. This one targets internal ops/SRE — an emerging, less crowded niche that proves you can apply agentic reasoning to infrastructure problems, not just chat-shaped ones.

Goals

Agent produces a ranked list of hypotheses with supporting evidence, not just one guess.
Any proposed fix is tested in a sandbox before being presented as a recommendation.
Full diagnostic trace is human-readable and useful even when the agent is wrong.

Non-Goals

Automatically applying fixes to production pipelines (always human-approved).
Covering arbitrary pipeline frameworks — pick one (e.g., a simple Airflow DAG or dbt project) for v1.

Users & Personas

Persona	Needs
Data engineer	Gets a ranked, evidence-backed root-cause diagnosis instead of starting from scratch

Tech Stack

Layer	Choice	Notes
Language/runtime	Python 3.11+	
Orchestration	LangGraph hypothesis-generation loop with an explicit "test before recommend" gate node	The sandbox-test step must be a hard gate, not a prompt instruction
Backend API	FastAPI	
LLM provider	OpenAI or Anthropic API	
Sandbox environment	Dockerized replay instance of a small target pipeline (e.g., a minimal Airflow DAG or dbt project)	Never touches production; this is the core safety boundary

Layer	Choice	Notes
Data store	PostgreSQL	Incidents, hypotheses (including disproven ones), test results
Async/background jobs	Celery + Redis	Sandbox test execution jobs
Observability & eval	Hypothesis-accuracy benchmark against injected failures; full diagnostic trace logging	Success metric is % of injected failures correctly root-caused
CI/CD	GitHub Actions	
Testing	Pytest + a deliberately-injected-failure benchmark suite	You author the failures, so ground truth is known
Deployment	Docker Compose (agent service + sandboxed pipeline environment together)	

Functional Requirements

- FR1: Ingest pipeline failure logs and recent change history (schema diffs, code diffs).
- FR2: Generate a ranked list of hypotheses for the failure cause.
- FR3: Test top hypotheses against a sandboxed replay of the pipeline (not production).
- FR4: Produce a structured incident report: hypothesis, evidence, test result, recommended fix.
- FR5: Log every hypothesis tested, including ones that were disproven.

Non-Functional Requirements

Category	Requirement
Safety	No agent action touches production data or systems directly
Explainability	Every hypothesis has supporting evidence cited from logs/diffs
Reproducibility	Sandbox test results are re-runnable and deterministic where possible

Architecture Notes

Log-parsing tool + schema/code-diff tool + sandboxed pipeline-replay environment (e.g., a Dockerized test instance of your target pipeline framework); hypothesis-generation loop with an explicit "test before recommend" gate.

API Surface (representative)



POST /incidents
GET /incidents/:id/hypotheses
GET /incidents/:id/report

Milestones

Log ingestion + manual hypothesis generation (no testing yet).
Sandboxed replay environment for hypothesis testing.
Full hypothesis-rank-test-report loop.
Seed a set of deliberately injected pipeline failures for benchmarking.
Report accuracy across the injected-failure benchmark set.

Success Metrics

% of injected failures where the top-ranked hypothesis was correct.
Mean number of hypotheses tested before finding the correct one.
Time-to-diagnosis vs. a manual baseline estimate.

Risks

Building a realistic-enough sandbox pipeline to inject meaningful failures into is the hardest part — keep the target pipeline intentionally small and well-understood.

PRD 6 — Sales Outreach Qualification Agent

Overview

An agent that enriches inbound leads, scores them against a qualification rubric, and drafts a personalized first-touch outreach message for human approval.

Problem Statement

Sales-agent verticals are heavily funded in the 2026 YC cohort. This project proves you can build a lead-to-outreach pipeline with structured scoring and controllable tone — not just “call an LLM to write an email.”

Goals

Qualification scoring is a structured, explainable tool call — not a black-box LLM judgment.
Drafts are persona/tone-controllable and require human approval before send (v1 safety gate).
Full pipeline runs on realistic (public or self-submitted) lead data.

Non-Goals

Actually sending emails autonomously (human-approved for v1).

CRM integration beyond a simple structured store.

Users & Personas

Persona	Needs
Sales rep	Reviews qualified leads with scores and approves/edits drafts before send

Tech Stack

Layer	Choice	Notes
Language/runtime	Python 3.11+	
Orchestration	Plain tool-calling pipeline (enrich → score → draft); LangGraph optional, not required for this linear flow	Don't over-engineer orchestration for a 3-step pipeline
Backend API	FastAPI	
LLM provider	OpenAI or Anthropic API	Used for qualitative sub-scores and draft generation only — the rubric itself is rule-based
Enrichment	A public company-data API wrapped in an adapter with rate-limit/backoff handling	Per external-integration adapter pattern
Data store	PostgreSQL	Leads, structured scores, drafts, approval state
Async/background jobs	Celery + Redis	Enrichment calls, rate-limited
Observability & eval	Score-correlation dashboard against a labeled "good lead" test set	Explainability requirement: score breakdown by rubric criterion, not one number
CI/CD	GitHub Actions	
Testing	Pytest + a labeled lead-quality eval set	
Deployment	Docker + a simple review/approval UI (Streamlit or Next.js)	Send action always gated behind human approval for v1

Functional Requirements

FR1: Ingest a lead (from a form submission or public dataset) with basic company/contact info.

FR2: Enrich the lead via public data sources (company size, industry, etc.) as a tool call.
FR3: Score the lead against a defined, structured qualification rubric.
FR4: Generate a personalized outreach draft with configurable tone/persona.
FR5: Human-approval gate before any message is marked "ready to send."

Non-Functional Requirements

Category	Requirement
Explainability	Qualification score is broken down by rubric criterion, not a single opaque number
Personalization quality	Drafts reference specific enriched facts about the lead, not generic templates
Rate limits	Enrichment API calls respect provider rate limits with backoff

Architecture Notes

Enrichment tool (public API/company-data lookup) → structured scoring tool (rule-based rubric, LLM only for qualitative sub-scores) → draft-generation call with persona parameter → human-approval queue.

API Surface (representative)

POST /leads GET /leads/:id/score POST /leads/:id/draft POST /leads/:id/approve

Milestones

Lead ingestion + manual scoring rubric (no enrichment yet).
Enrichment tool integration.
Structured scoring + draft generation.
Persona/tone control + human-approval workflow.
Benchmark: qualification score correlation against a labeled "good lead" test set.

Success Metrics

Correlation between agent qualification score and human-labeled lead quality on a test set.
% of drafts approved without edits vs. requiring rework.
Enrichment success/failure rate.

Risks

Public lead-enrichment data sources can be inconsistent — build a fallback path for leads with sparse enrichment data.

PRD 7 — Meeting-to-Action-Items Agent

Overview

An agent that turns a meeting transcript into structured, assigned action items and tracks follow-through across subsequent meetings — the one project in this set explicitly designed to test long-lived state, not single-session logic.

Problem Statement

Most agent demos are one-shot: input in, output out, session forgotten. This project proves you can design agents with persistent memory that reasons across multiple sessions over time — a materially harder and rarer skill.

Goals

- Correctly extract action items with assignees from a raw transcript.
- Track each action item's status across future meetings (open, in-progress, done, stale).
- Surface stale/overdue action items proactively in later meeting summaries.

Non-Goals

- Real-time transcription (assume transcripts are provided as text input).
- Calendar/task-tool integration beyond your own persistence layer for v1.

Users & Personas

Persona	Needs
Meeting participant	Gets a clean action-item summary and follow-up tracking without manual note-taking
Team lead	Sees which action items are stale across a series of meetings

Tech Stack

Layer	Choice	Notes
Language/runtime	Python 3.11+	
Orchestration	Structured-output extraction call → cross-reference logic against persisted state	This is the one project where persistence design matters more than orchestration framework choice

Layer	Choice	Notes
Backend API	FastAPI	
LLM provider	OpenAI or Anthropic API, using structured/JSON output mode	Extraction accuracy is the core metric — enforce schema validation on output
Data store	PostgreSQL	Action items keyed by meeting-series ID; this is the system of record for cross-session state
Async/background jobs	Not required for v1	Per-transcript processing is fast enough to run synchronously
Observability & eval	Extraction precision/recall tracking + a state-consistency checker (detects silently lost items)	
CI/CD	GitHub Actions	
Testing	Pytest + a synthetic 4–6 transcript multi-meeting eval series	Ground truth authored by you since no real recurring-meeting data is assumed
Deployment	Docker + a simple dashboard UI showing action-item status over time	

Functional Requirements

- FR1: Ingest a meeting transcript and extract structured action items (task, assignee, due date if mentioned).
- FR2: Persist action items linked to a person and a recurring meeting series.
- FR3: On a new transcript for the same series, cross-reference previously extracted open items and detect follow-through (mentioned as done, still open, or not mentioned = potentially stale).
- FR4: Generate a meeting summary that includes both new action items and status updates on prior ones.
- FR5: Flag action items open for more than N meetings as stale.

Non-Functional Requirements

Category	Requirement
State correctness	Action-item state persists accurately across sessions; no silent loss
Extraction quality	Assignee/task extraction accuracy measured against labeled transcripts
Explainability	Stale-item flags reference exactly which past meeting they originated from

Architecture Notes

Extraction call (structured output) → persistence in Postgres keyed by meeting-series ID → cross-reference step on each new transcript against open items for that series → summary-generation call combining new + carried-over items.

API Surface (representative)

```
POST /meetings/:series_id/transcript
GET /meetings/:series_id/action-items
GET /meetings/:series_id/summary
```

Milestones

- Single-transcript extraction only (no cross-session state).
- Persistence layer + action-item tracking across a meeting series.
- Cross-reference logic for status updates on prior items.
- Staleness detection and flagging.
- Evaluation against a labeled multi-meeting transcript set (can be synthetic/self-generated).

Success Metrics

- Action-item extraction precision/recall against labeled transcripts.
- % of stale items correctly flagged vs. missed.
- State-consistency check: 0 silently lost action items across a multi-meeting test run.

Risks

Needs a multi-meeting transcript series to test cross-session logic meaningfully — plan to generate a synthetic but realistic series (e.g., 4–6 linked transcripts) if you don't have real recurring-meeting data.

PRD 8 — Multi-Source Enterprise Knowledge Assistant

Overview

A RAG system that answers questions by retrieving from heterogeneous sources — chat exports, wiki pages, and code — with a router deciding which source(s) to query per question.

Problem Statement

Real enterprises never have one clean document store. Naive single-index RAG breaks down when sources have different structure and reliability. This project proves you can handle that mess with a source-aware retrieval design.

Goals

Router correctly selects relevant source type(s) per question, measured against a labeled test set.
Citations clearly indicate which source type an answer came from.
Retrieval quality is comparable to or better than a single-index baseline, measured explicitly.

Non-Goals

Real-time sync with live enterprise systems (batch ingestion is fine for v1).
More than 3 source types for v1.

Users & Personas

Persona	Needs
Employee	Asks a question, gets an answer grounded in the right internal source(s)

Tech Stack

Layer	Choice	Notes
Language/runtime	Python 3.11+	Direct extension of <code>production-grade-rag-engine</code>
Orchestration	A router step (rule-based or a small classification model) + source-specific retrieval	Router should be cheap/fast — don't use a frontier-model call just to pick a source type
Backend API	FastAPI	Reuses your existing RAG engine's API conventions
LLM provider	Gemini or another provider via adapter, matching your existing repo	Keep provider consistent with your existing production RAG engine for a coherent portfolio story
Retrieval / vector DB	Qdrant, with either separate collections per source type or metadata-filtered single collection	Decision point worth documenting as an ADR-style note in the README

Layer	Choice	Notes
Data store	PostgreSQL	Source metadata, ingestion state, query logs
Async/background jobs	Celery + Redis (or Inngest, matching your existing repo)	Per-source-type ingestion pipelines
Observability & eval	RAGAS-style harness reporting precision@5 per source type + router accuracy against a labeled set	This is the project's central benchmark — multi-source vs. single-index baseline
CI/CD	GitHub Actions	
Testing	Pytest + a 40–60 question labeled routing test set	
Deployment	Docker + Streamlit query UI	

Functional Requirements

FR1: Ingest and index 3 distinct source types (e.g., Slack-style export, wiki pages, codebase) with source-appropriate chunking per type.

FR2: A router step classifies which source type(s) are likely relevant to a given question.

FR3: Retrieve from the selected source(s) and generate a cited answer, with citations tagged by source type.

FR4: Fall back to querying all sources if the router is low-confidence.

Non-Functional Requirements

Category	Requirement
Retrieval quality	Precision@5 measured per source type and against a single-index baseline
Router accuracy	Measured against a labeled "which source(s) should answer this" test set
Citation clarity	Every answer's citations indicate source type unambiguously

Architecture Notes

Separate ingestion/chunking pipelines per source type, separate vector indexes (or a single index with source-type metadata filtering), a lightweight classification step (rule-based or small-model) for routing, unified citation formatting layer.

API Surface (representative)


POST /query
GET /query/:id/sources

Milestones

Single-source baseline RAG (e.g., wiki only).
Add second and third source types with type-specific chunking.
Router implementation + labeled routing test set.
Unified citation formatting across source types.
Benchmark: multi-source vs. single-index baseline retrieval quality.

Success Metrics

Router accuracy (%) on labeled test questions.
Precision@5 per source type, and overall vs. single-index baseline.
% of answers with correctly labeled source-type citations.

Risks

Constructing a labeled "correct source(s) per question" test set is manual — a 40–60 question set is enough to be credible.

PRD 9 — Long-Document / Legal Contract RAG

Overview

A RAG system purpose-built for long documents (100+ pages) where answers require correct cross-referencing across multiple sections/clauses — a distinct, harder problem than short-document RAG.

Problem Statement

Naive chunking breaks on long documents: answers that depend on connecting Section 3 to Section 47 get missed entirely. This project proves you understand hierarchical and multi-hop retrieval, a meaningfully harder RAG skill than the standard short-doc case.

Goals

Correctly answer questions that require connecting information across non-adjacent sections.
Retrieval respects document structure (section/clause boundaries), not fixed-size chunks alone.
Explicit before/after comparison against naive fixed-size chunking.

Non-Goals

OCR/scanned-document handling (assume clean text/PDF-extractable documents for v1).
Multi-document cross-referencing (single long document at a time for v1).

Users & Personas

Persona	Needs
Contract reviewer	Asks questions requiring cross-referenced clauses, gets a correctly connected answer

Tech Stack

Layer	Choice	Notes
Language/runtime	Python 3.11+	
Orchestration	Hierarchical chunker (heading/numbering-aware) → two-stage retrieval (initial hop + cross-reference expansion hop)	The multi-hop expansion logic is the differentiator — keep it as an explicit, inspectable step, not buried in a single retrieval call
Backend API	FastAPI	
LLM provider	OpenAI or Anthropic API	
Retrieval / vector DB	Qdrant, with section/clause-level metadata attached to each chunk	
Cross-reference detection	Regex/rule-based pattern matching (e.g., "Section X.Y") feeding retrieval expansion	Deterministic, not LLM-based — keeps this step fast and auditable
Data store	PostgreSQL	Document structure, detected cross-reference graph
Async/background jobs	Celery + Redis	Document ingestion/chunking pipeline
Observability & eval	Multi-hop question accuracy benchmark, hierarchical vs. fixed-size-chunking baseline, reported as an explicit delta	
CI/CD	GitHub Actions	
Testing	Pytest + a labeled multi-hop question set built against a real cross-reference-heavy public document	

Layer	Choice	Notes
Deployment	Docker + a simple query UI	

Functional Requirements

- FR1: Ingest a long document and chunk it hierarchically (document → section → clause), preserving structural metadata.
- FR2: Build a multi-hop retrieval step: first retrieve a candidate section, then expand to related/cross-referenced sections before generating an answer.
- FR3: Detect explicit cross-references within the text (e.g., "as defined in Section 4.2") and use them to inform retrieval expansion.
- FR4: Generate answers with citations at the clause level, not just document level.

Non-Functional Requirements

Category	Requirement
Retrieval quality	Measured accuracy on multi-hop questions vs. a fixed-size-chunking baseline
Structural fidelity	Chunk metadata correctly preserves section/clause hierarchy
Latency	Multi-hop retrieval p95 within an acceptable interactive budget (e.g., 5s)

Architecture Notes

Hierarchical chunker parsing document structure (headings/numbering), a retrieval graph or two-stage retrieval (initial + expansion hop) informed by detected cross-reference patterns, clause-level citation formatting.

API Surface (representative)


POST /documents
POST /documents/:id/query

Milestones

- Baseline: fixed-size chunking RAG (for comparison).
- Hierarchical chunking preserving section/clause structure.
- Cross-reference detection + multi-hop retrieval expansion.
- Clause-level citation generation.
- Benchmark: multi-hop question accuracy, hierarchical vs. fixed-size baseline.

Success Metrics

Accuracy on a labeled set of multi-hop questions (hierarchical vs. baseline comparison, reported as a delta).

Clause-level citation precision.

Latency p95 for multi-hop queries.

Risks

Finding or constructing a genuinely long, cross-reference-heavy public document (e.g., a public contract template, terms of service, or regulation text) is important — pick one with real internal cross-references, not just length.

PRD 10 — Code-Aware RAG for a Real Codebase

Overview

A RAG system over a real codebase that understands code structure (functions, imports, call graphs) rather than treating code as plain text — directly complementary to your existing production-grade-rag-engine repo.

Problem Statement

Text-similarity retrieval over code misses structurally relevant context (e.g., "how does X work" needs the function definition, its callers, and its docstring — not just text near the word "X"). Dev-tools AI companies specifically test for this distinction.

Goals

Retrieval understands code structure: functions, classes, imports, call relationships.

Answers combine code + docstrings + relevant commit history as distinct, separately retrievable channels.

Demonstrate clear retrieval-quality improvement over naive text-chunked code RAG.

Non-Goals

Multi-repo/cross-repo retrieval (single real codebase for v1).

Auto-generating code changes (Q&A/explanation only, not code editing, for v1).

Users & Personas

Persona	Needs
Developer Asks "how does X work" or "where is Y used" and gets structurally correct, cited answers	

Tech Stack

Layer	Choice	Notes
Language/runtime	Python 3.11+	
Orchestration	AST parser (tree-sitter, language-appropriate) → symbol-graph construction → two-stage retrieval (vector similarity + graph expansion)	Directly extends <u>production-grade-rag-engine</u> with structural awareness
Backend API	FastAPI	
LLM provider	Gemini or another provider via adapter, matching your existing repo	Keeps stack consistent with your existing RAG engine for a coherent portfolio narrative
Retrieval / vector DB	Qdrant for code-chunk embeddings	
Symbol graph store	PostgreSQL adjacency table (nodes = functions/classes, edges = calls/imports) — a dedicated graph DB (e.g., Neo4j) is a documented, justified upgrade if traversal complexity grows	Start simple per blueprint-style "evidence-based scaling" thinking — don't reach for a graph DB before proving you need one
Commit history index	Separate Qdrant collection or metadata-filtered index, queried conditionally	
Data store	PostgreSQL	Symbol graph, indexing job state
Async/background jobs	Celery + Redis	Repo-indexing jobs
Observability & eval	Precision@5 benchmark vs. a naive text-chunked baseline on the same codebase	Central benchmark for this project
CI/CD	GitHub Actions	
Testing	Pytest + a labeled "how does X work" question set built against your own repos	Your own existing repos are the natural, ready-made test corpus
Deployment	Docker + a CLI or simple query UI	

Functional Requirements

FR1: Parse the target codebase into an AST-informed representation; chunk by function/class rather than fixed line ranges.

FR2: Build a symbol graph capturing function definitions, call sites, and imports.

FR3: Retrieval combines semantic similarity with symbol-graph traversal (e.g., "also include callers of this function").

FR4: Separately index and retrieve from commit history/docstrings as an additional evidence channel when relevant.

FR5: Answers cite exact file/function/line references.

Non-Functional Requirements

Category	Requirement
Retrieval quality	Precision@5 measured against a naive text-chunked baseline on the same codebase
Structural correctness	Symbol-graph traversal returns genuinely related code (verified against a labeled set)
Latency	p95 query latency within an interactive budget

Architecture Notes

AST parser (language-appropriate, e.g., tree-sitter) for function/class-level chunking, a symbol graph (nodes = functions/classes, edges = calls/imports) stored alongside vector embeddings, a two-stage retrieval combining vector similarity + graph expansion, separate commit-history index queried conditionally.

API Surface (representative)

POST /repo/index POST /repo/query
--

Milestones

Baseline: naive text-chunked RAG over the codebase (for comparison).

AST-informed function/class-level chunking.

Symbol-graph construction + graph-informed retrieval expansion.

Commit-history/docstring channel added.

Benchmark: precision@5 vs. baseline on a labeled "how does X work" question set.

Success Metrics

Precision@5 improvement over naive text-chunked baseline (reported as a concrete delta).

% of answers with correct file/function/line citations.

Latency p95.

Risks

Pick a codebase you know well enough to hand-label a meaningful "correct answer" test set — your own existing repos are a good, ready-made choice here.

Cross-Project Notes

Projects 1-7 (agentic) and 8-10 (RAG) are designed to be built on the same underlying infrastructure where possible — reuse one LangGraph orchestration setup, one Postgres schema convention, and one tracing/observability layer across all of them (mirroring the "shared internal platform" story from your engineering-blueprint PRDs). This also sets you up cleanly for Project #13 (the standalone eval/red-team framework) to be applied uniformly across whichever of these ten you actually build.